

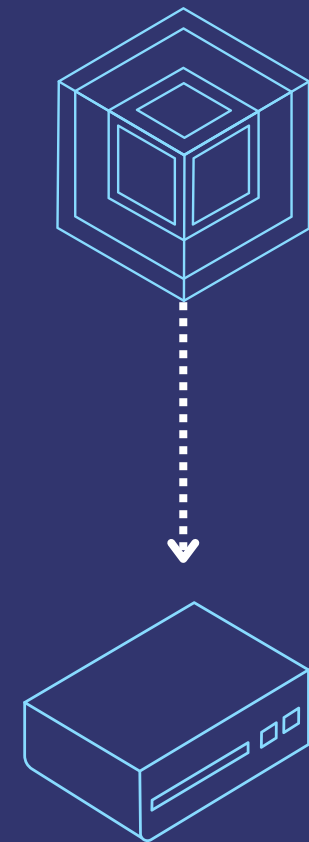
Avaliação de Disponibilidade e Tolerância a Falhas em MongoDB Replicado

Sistemas Distribuídos

Arthur Moro
Eduardo Spat
Fabrício Thomas
Luis Henrique Cardoso

O sistema antigo

1 único banco, Mongo ou Postgres



Aplicação: Node.js conectada ao banco

Não fazia uso de
réplicas de bancos,
nem das principais
ferramentas do
NoSQL:

- Crud totalmente estruturado
- Schema não adaptado para utilização de documentos

O sistema antigo

Resultados

- O banco SQL teve desempenho superior em todas as tarefas e experimentos, mas não por um defeito do NoSQL, mas um problema de arquitetura

O novo experimento

- Foca em utilizar melhor os recursos disponíveis do NoSQL, para resolver os problemas percebidos

Objetivo

O que foi feito?

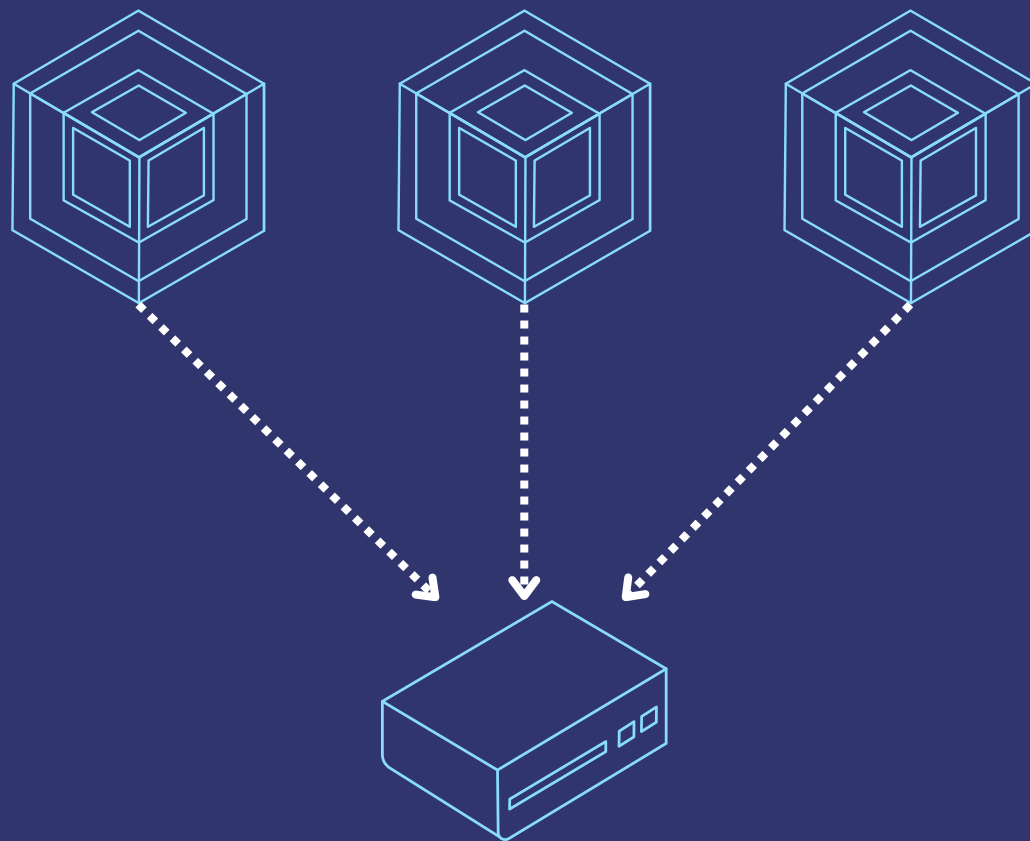
- Implementação 3 réplicas de NoSQL
- Desenvolvimento de uma API (Node.js) para gerenciar pedidos.
- Criação de um "Chaos Monkey" para injetar falhas reais na infraestrutura.

O que queremos provar?

- Como o sistema se comporta quando um nó cai sem aviso
- A diferença entre disponibilidade de escrita vs. leitura.

Arquitetura do Sistema

3 Nós de Banco: mongo1, mongo2, mongo3. ReplicaSet



Aplicação: Node.js conectada via driver Mongoose.



Load Tester Script
Python disparando
carga concorrente.



Chaos Monkey
Derruba e revive nós



Seed para gerar
50.000+ documentos

Chaos Monkey



Modo Aleatório

- Derruba um dos nós (mongo1/2/3)
- Espera 10s
- Liga de novo



Modo Caça Lider

- Detecta nó primário
- Mata o primário propositalmente
- ReplicaSet inicia nova eleição automaticamente

Configurações de Consistência

► **WRITE_CONCERN: 1 ou Majority**

1: A confirmação de escrita vem apenas do Líder (mais rápido, menos seguro contra perdas se o líder cair instantaneamente). Ponto único de falha.

Majority: leitura só é confirmada quando a maioria dos nodos aceitam. É mais lento mas mais resiliente.

► **CHAOS_FAILURE_CHANCE= '0.0 - 1.0'**

Chance um dos 3 nodes cair de 0% a 100%

► **READ_PREFERENCE: secondaryPreferred ou primary**

SecondaryPreferred: A aplicação tenta ler dos nós Secundários. Se eles caírem, lê do Primário.

Primary: lê do primário, caso ele caia, causa falha de leitura.

► **CHAOS_KILL_LEADER= 'False - True'**

True = Sempre descobre e mata o líder.

False = Mata um nó aleatório.

Configurações de loadtester

► **DURATION = "45s"**

Duração em segundos do teste.

► **CONCURRENCY=10**

Quantidade de threads enviando requisições simultaneamente.

► **READ_RATIO = 0.0 - 1.0**

Proporção de leituras, o resto vai para escritas.

Ex: READ_RATIO = 0.8 - 80% leituras/20% escritas.

Metodologia de Teste

Script Python *loadtester*

Cenário do Teste

Duração: 45 segundos.

10 threads mandando requisições

80% de leitura

Chance de 100% de falha neste teste

Variável	Valor Configurado
READ_PREFERENCE	secondaryPreferred
WRITE_CONCERN	1
CHAOS_FAILURE_CHANCE	1.0
DATASET_SIZE	10000
COMPLEX_DOC_RATIO	0.6

Resultados

Tabela Resumo

- Leituras (GET): 3366 requisições | 0 Erros.
- Escritas (POST): 872 requisições | 150 Erros.

[illegible]

Conclusão

Quando o líder morre, o cluster entra em eleição e ninguém aceita escritas, por conta do "**WRITE_CONCERN = 1**" fazer com que apenas o líder possa confirmar escritas. Isso faz com que haja erros por parte das escritas

Porém, como configuramos "**READ_PREFERENCE = secondaryPreferred**", as leituras (quadrados verdes) continuaram fluindo pelos nós sobreviventes.

Metodologia de Teste

Cenário do Teste

Parâmetro	Valor
Preferência de leitura	secondaryPreferred
Responsabilidade de escrita	majority
Chance de falha	0.3 ou 30%
Matar líder	FALSO
Taxa de leitura	0.95 ou 95%

Resultado esperado

O sistema, em geral, ganha muita disponibilidade, pois distribui as leituras entre os nós devido ao **secondaryPreferred**

A escrita majoritária também garante confiabilidade dos dados, ainda que custe um pouco de performance

A queda de um nó, mesmo que o líder, provavelmente não causará impacto significativo. 30% de falha ainda é um número alto, mas muito mais realista que os 100% do teste anterior.